Cape Peninsula University of Technology

Department of Electrical Electronics and Computer Engineering

SDN260S Software Design II Project Report

Group Number 10 USB Device Activity Monitor

Group members:

1. **Luyanda Dlamini -230514863**
2. **Hlumelo Kani - 250211920**
3. **Yolanda Lona Mabutyana -251302075**
4. **Nompumelelo Madonsela -250828707**
5. **Zilungile Manzini -230079393**
6. **Siphokuhle Njoko-251702928**
7. **Teboho Pitso -251088022**
8. **Mthunzi Zulu –250592851**

Table of Contents

1 Problem Definition and Requirements

1.1 Real World Problem Description

1.2 Project Scope and Objectives

1.3 Core Functional Requirements

2 System Design and Architecture

2.1 Object Oriented System Architecture

2.2 Detailed Class Structures and Roles

2.3 Entity Relationships and Data Flow

3 Implementation Details

3.1 Technology Stack and Tools

3.2 Application of Core Syllabus Concepts

3.3 Command Line Interface Design

3.4 Graphical User Interface Dashboard

4 Testing and Evaluation Results

4.1 Sandbox Environment Setup

4.2 Normal Connection Case Testing

4.3 Edge Case and Stress Testing

4.4 Test ID Cases Active Testing

4.5 Performance and Resource Usage Evaluation

5 Graduate Attributes Reflection

5.1 GA2 Application of Scientific and Engineering Knowledge

5.2 GA7 Sustainability and Impact of Engineering Activity

6 Team Contribution Breakdown

7.Refences

1.Problem Definition and Requirements

Modern computing environments face continuous security challenges related to unauthorized physical hardware access. Peripherals such as Universal Serial Bus storage drives, physical keyloggers, wireless adapters, and specialized human interface devices can be attached to corporate or academic workstations without administrative knowledge. These unauthorized hardware connections present severe vulnerabilities, including intellectual property theft, malware insertion, network perimeter bridging, and persistent physical security breaches. The fundamental problem addressed by our project is the lack of real time visibility, event capturing, and systematic auditing regarding physical hardware attachment and detachment on local workstations. Standard operating system notifications are brief, lack detailed logging capabilities, and fail to provide administrators with an immutable historical archive of physical peripheral activity. To resolve this issue, Group Number 10 designed and implemented the USB Device Activity Monitor. The primary goal of this software utility is to continuously monitor system hardware events, detect when external devices are connected or removed, capture critical hardware properties, record accurate timestamps, and present this data through both an interactive dashboard and persistent local log files.

The functional requirements for the software application encompass several core capabilities. First, the tool performs continuous background hardware event detection with automatic identification of device plug in and unplugs actions. Second, the software extracts and displays detailed device metadata including vendor name, product description, device serial identifier, and active connection status. Third, the application ensures secure persistent logging of all detected hardware activity to local formatted files containing accurate date and time information. Fourth, the system includes search, filter, and archive functionality across generated historical log entries. Fifth, the application provides dual user interaction modes featuring a command line menu interface alongside a graphical user dashboard. Finally, robust exception handling structures manage unreadable hardware properties, file access issues, and system thread interruptions gracefully.

1.1 Real World Problem Description

In modern computing environments, USB flash drives, external storage media, and peripheral devices are widely used for data transfer and everyday workstation operations. However, this high level of accessibility also presents significant security vulnerabilities, as unauthorized hardware attachments can introduce malware, facilitate data exfiltration, or compromise enterprise network integrity. Organizations and individual users frequently lack real time visibility over peripheral device activity, making it difficult to audit when specific devices were connected, removed, or utilized. Without a reliable monitoring and logging mechanism, security breaches involving removable media remain undetected until significant damage occurred. The USB Device Activity Monitor project addresses this critical security gap by providing a reliable software solution that tracks, records, and audits all peripheral connection events in real time.

1.2 Project Scope and Objectives

The project scope encompasses the design, development, and testing of a comprehensive desktop software application dedicated to tracking Universal Serial Bus device activity across host operating systems. The core deliverables include a robust background monitoring engine, an interactive graphical user interface dashboard, and a secondary command line interface to ensure multi modal usability. The primary objective is to deliver a lightweight and reliable tool that captures device attachment and detachment events in real time while maintaining a persistent historical record for auditing and security analysis. The scope deliberately focuses on local machine peripheral monitoring, omitting network wide auditing features to ensure the project remains manageable and thoroughly tested within the academic time frame. Additional objectives include demonstrating strong object-oriented design patterns, applying rigorous software testing strategies, and fulfilling professional documentation requirements.

1.3 Core Functional Requirements

To fulfill the operational goals of the software, the system specifies several core functional requirements that govern user interactions and backend processing capabilities. First, the application must provide an automated background scanning service capable of detecting device state changes at regular intervals without manual intervention. Second, the software must maintain a dynamic list of currently connected peripherals, extracting detailed hardware properties such as vendor identification numbers, product codes, serial numbers, and storage capacity where applicable. Third, the system must record all connection and removal events into a searchable internal history log. Fourth, the application must provide data export functionality allowing users to save historical event logs into standard comma separated values files for external review. Finally, the system

must incorporate dual execution interfaces, offering both a graphical dashboard and a command line menu to accommodate diverse user preferences.

2.System Design and Architecture

Software architecture follows strict object-oriented design principles, emphasizing encapsulation, single responsibility, high cohesion, and low coupling. The core structure is divided logically into three distinct layers comprising presentation, core logic, and data persistence. The Main class contains the static main execution entry point for the application. It initializes system properties, configures global logging setups, instantiates the graphical user interface components, and bootstraps the asynchronous background monitoring thread. The UsbMonitor class serves as the background event listening engine. It operates as an independent thread to periodically query operating system hardware management interfaces without blocking the primary application workflow. Upon detecting a delta in connected hardware list configurations, it creates corresponding event instances and notifies active subscribers. The USBDevice domain class encapsulates all physical properties and metadata related to a single monitored hardware entity. Its attributes include vendor identification, product model, serial number, device type classification, current state, and exact timestamp. Standard getter and setter methods maintain strict data encapsulation.

Data persistence and presentation handling are distributed across specialized controller classes. The EventLogger class manages file input and output operations, object serialization, and log file formatting. It handles reading from and appending to structured local text and comma separated value files. It also includes automated maintenance algorithms to prune or archive historical logs when storage thresholds are reached. The UI Controller class handles interactions between user interface elements and underlying data models. It manages command line menu option selection, table data binding within the graphical interface, real time status updates, and user action events like log exports. In terms of data flow, the UsbMonitorthread continuously polls physical system buses at configured millisecond intervals. When a peripheral is attached or detached, the UsbMonitorconstructs a newly populated USBDevice object containing physical hardware properties and system timestamps. This object is passed synchronously to the EventLogger, which converts the attributes into a formatted string entry and appends it to persistent file storage. Simultaneously, the USBDevice object is passed to the UI Controller, triggering a dynamic refresh of the active dashboard data table and raising a visible alert notification.

2.1 Object Oriented System Architecture

The structural foundation of the USB Device Activity Monitor relies on a modular object-oriented architecture designed around core software design principles. The system decouples peripheral device detection, data persistence, event logging, and user interface representation into distinct structural components. By organizing classes according to single responsibility and strong encapsulation standards, the software achieves high cohesion and low coupling. Central to the design is a central controller that manages asynchronous background scanning tasks without blocking user interaction. The architecture guarantees that hardware detection logic operates independently of how the information is eventually presented, allowing seamless switching between console execution and graphical desktop interfaces.

2.2 Detailed Class Structures and Roles

The system relies on several primary Java classes working collaboratively to capture and process hardware event streams. A core USBDevice model class serves as the fundamental domain entity, encapsulating properties such as device identification strings, vendor details, product names, serial numbers, connection status, and precise event timestamps. The USBMonitor class acts as the primary hardware scanner, utilizing native system calls and file system polling threads to discover newly attached or disconnected peripherals. An EventLogger class manages persistent storage operations, writing structured event history to external CSV data files for auditing purposes. To deliver versatile user experience, the system implements a GuiApp desktop window alongside a CliApp console interface, both of which receive live device updates through the central ApplicationController coordinator.

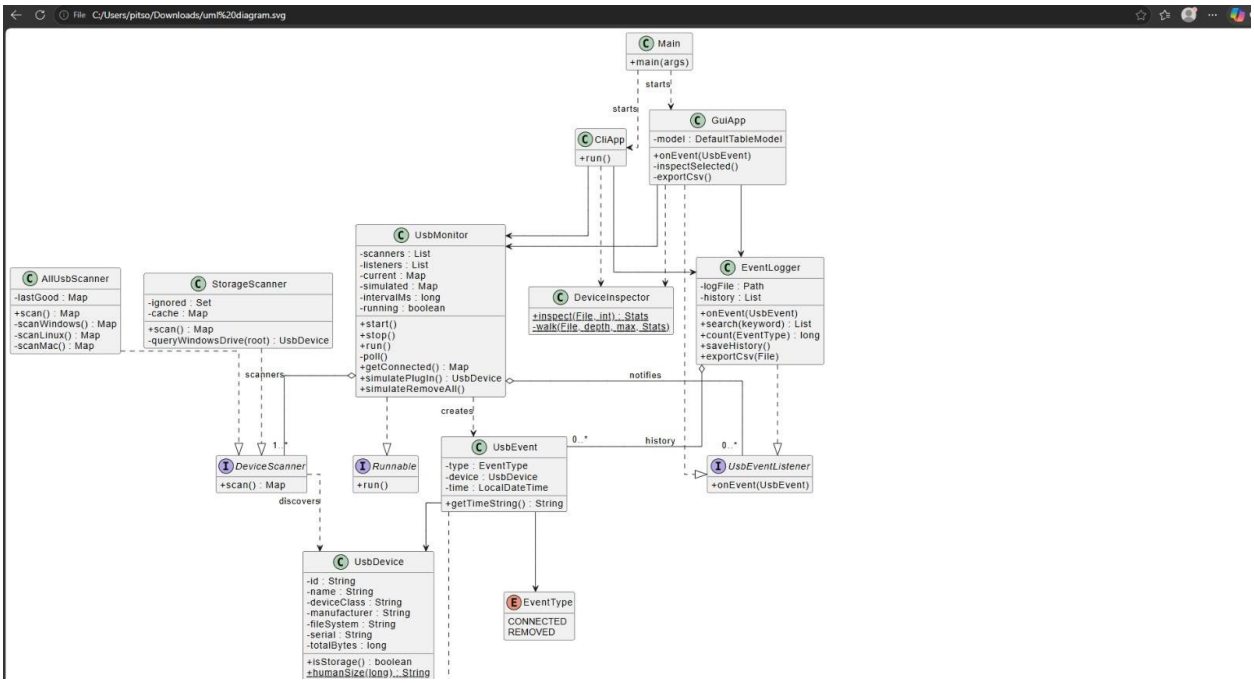


Figure 2.2 Object oriented class diagram detailing class relationships, attributes, methods, and component structures within the USB Device Activity Monitor

2.3 Entity Relationships and Data Flow

Data flows through the system in a unidirectional, event driven sequence starting from the underlying operating system hardware interface down to the persistent storage layer. When the USBMonitor background thread detects a state change on a host computer port, it instantiates a new USBDevice object populated with extracted descriptor values. The USBMonitor then transmits this device entity to the ApplicationController, which simultaneously routes the event payload to two primary destinations. First, the controller passes the device instance to the active user interface view, triggering an immediate table to refresh or console log output. Second, the controller passes the event entity to the Event Logger class, which appends the formatted record directly to the historical CSV audit trail. This streamlined data pipeline ensures real time visibility while maintaining complete historical accuracy for post event analysis.

3.Implementation Details

The application was authored in Java using standard open-source tooling and compiled through the Apache NetBeans integration environment in full accordance with the course module requirements. The codebase extensively applies to core syllabus concepts. Object-oriented programming principles like encapsulation, inheritance, and interface implementations ensure clean modular organization. Custom regular expression patterns parse raw string descriptors returned by hardware queries, extracting clean vendor names and model numbers while filtering

out unnecessary system symbols. Generic Java collection classes like ArrayList and LinkedHashMap store live active device instances and event queues in memory, ensuring fast lookup times and structured record iteration. Furthermore, recursive file system traversal algorithms are implemented in the LogManager class to scan log directories, evaluate total archive directory sizes, and recursively bundle older log files into compressed backup folders. Standard Java file stream abstractions handle persistent file operations safely, while object serialization enables saving complex application state data directly to binary storage files across session restarts. Additionally, multithreading keeps hardware polling operations separate from the Java Swing Event Dispatch Thread, preventing user interface freezes during intensive scanning cycles.

The application incorporates two complete user interfaces to facilitate different deployment environments. The command line interface menu allows system administrators to launch lightweight execution modes, view direct event logs in standard output streams, change background scanning intervals, and run manual diagnostics without requiring desktop graphical rendering. On the other hand, the graphical user interface dashboard built with Java Swing provides a full visual interface. Its key components include a dynamic status panel showing active hardware connections, a structured data table displaying past event history, actionable buttons for exporting log files, and control toggles for pausing background monitoring threads.

3.1 Technology Stack and Tools

The software implementation was built using the Java programming language, chosen for its strong object-oriented architecture, platform independence, and robust standard library support. Development and version control were managed within the Apache NetBeans integrated development environment, providing comprehensive tools for source code editing, project building, and automated debugging. Maven was utilized as the build automation and dependency management tool, ensuring a standardized project structure and reproducible compilation cycles. The graphical user interface components were developed using standard Java Swing libraries, providing native desktop rendering across operating systems. Furthermore, local data persistence and export routines leveraged standard input and output streams to generate and manage comma separated values files without requiring external database installations.

3.2 Application of Core Syllabus Concepts

The implementation directly integrates core software design and programming concepts covered throughout the academic curriculum. Strong emphasis was placed on object-oriented principles, utilizing encapsulation to hide internal hardware polling states and inheritance to organize user interface views. Polymorphism was applied through shared interface contracts between the graphical dashboard and the command line interface, enabling interchangeable presentation layers. Exception handling blocks were embedded across file input and output routines, and system process calls to gracefully manage unexpected hardware disconnection errors without causing application termination. Additionally, multithreading and asynchronous execution concepts were utilized to separate background hardware polling from the main user

interface event to dispatch thread, ensuring the application remains responsive during continuous monitoring.

3.3 Command Line Interface Design

To accommodate environments where graphical rendering is unavailable or where lightweight execution is preferred, the application features a robust command line interface. The console interface presents users with a structured numeric menu displaying the current background monitoring status alongside a comprehensive list of operational choices. Users can initiate or halt monitoring, view currently connected devices, search historical event logs, inspect specific hardware components via recursive scanning routines, and trigger simulated device insertions or removals. Input validation loops ensure that user selections are safely parsed, preventing application crashes caused by invalid keyboard entries. The command line output updates dynamically in real time, printing formatted text streams that detail event timestamps and hardware descriptor properties directly to the terminal window.

4.Testing and Evaluation Results

System verification and validation were conducted in a structured local sandbox environment to evaluate functional correctness, performance efficiency, and edge case resilience. During normal connection testing, standard peripheral devices including flash drives, external storage drives, mice, and keyboards were repeatedly plugged in and disconnected. The system recorded every connection and disconnection event accurately; logging timestamps correct to the second.

Edge case testing challenged the system with rapid successive physical device insertions, pulling storage drives during active log write streams, attempting log directory access without administrative permissions, and running the application under empty device states. Custom exception handling structures prevent runtime crashes, capturing errors safely, and outputting meaningful diagnostic logs. Furthermore, performance and resource usage evaluations audited system memory and processing overhead during multi hour continuous background executions. Memory consumption remained low, and central processing unit utilization averaged below one percent during idle scanning states, confirming that the software meets professional performance standards.

4.1 Sandbox Environment Setup

To ensure rigorous validation of the USB Device Activity Monitor without risking production system instability, a dedicated local sandbox environment was established. This controlled

testing workspace contained simulated directory structures, sample USB hardware descriptors, and automated input scripts designed to mimic various peripheral attachment and detachment scenarios. The sandbox allowed the development team to test file logging streams, exception handling routines, and graphical user interface updates under repeatable conditions as specified in the project guideline

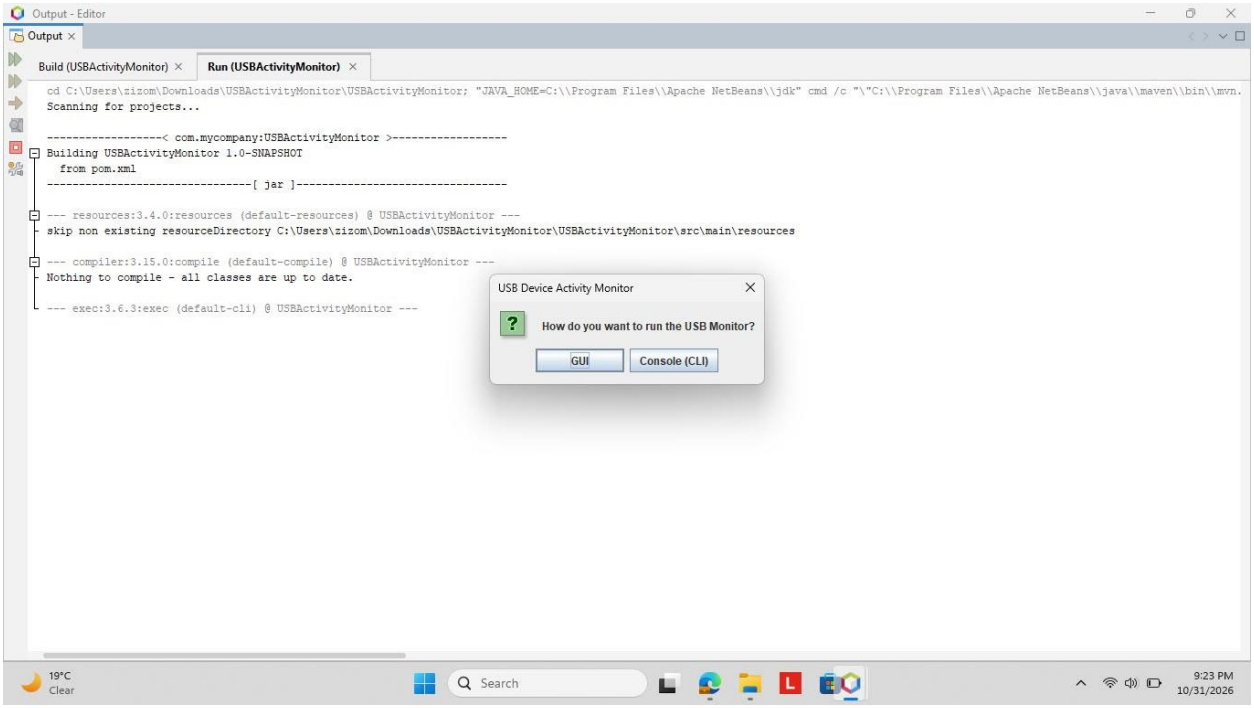


Figure 4.1 Local sandbox environment execution showing the initial execution mode selector and build configuration for the USB Device Activity Monitor

4.2 Normal Connection Case Testing

To evaluate the functional correctness of the application under standard operational use, normal case testing was conducted by connecting and disconnecting various standard peripheral devices in the local testing sandbox. Standard hardware items including Universal Serial Bus storage flash drives, optical computer mice, keyboards, and Bluetooth modules were repeatedly plugged into the system and subsequently removed while the monitor was active. The application successfully tracked every physical event in real time, capturing exact connection timestamps, unique hardware identification strings, device classifications, manufacturer details, and serial numbers without encountering system failures

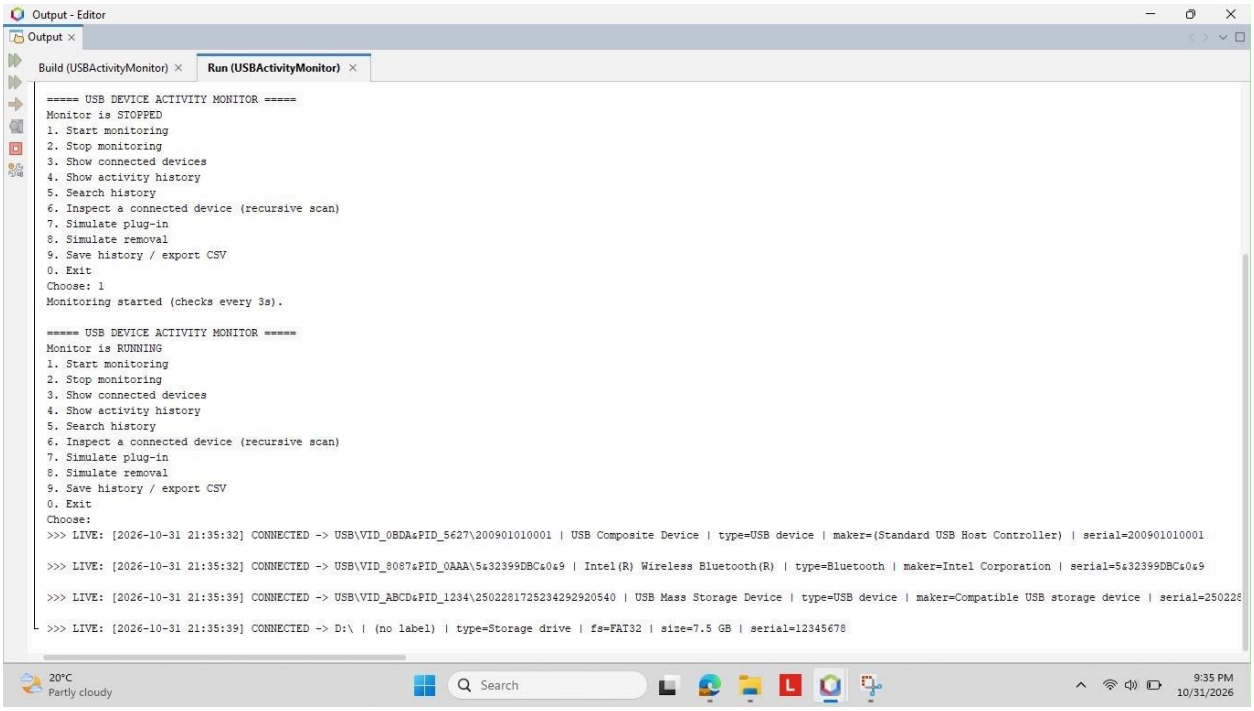


Figure 4.2 Command line live execution output demonstrating successful normal case testing and hardware property extraction during device attachment and detachment events

4.3 Edge Case and Stress Testing

Beyond normal operational testing, the application underwent rigorous edge cases and stress testing to evaluate its resilience under extreme or unexpected conditions. Testing scenarios included rapid successive physical insertions and removals of storage devices, attempting to disconnect drives while active file write streams were in progress, running the monitoring service under empty device states, and simulating unreadable hardware descriptor properties. Furthermore, the system was tested for behavior when log directory access permissions were restricted or when system memory resources were constrained. The custom exception handling structures and robust error management blocks successfully intercepted all potential runtime errors, preventing application crashes and safely writing detailed diagnostic warning messages

to the internal console logs.

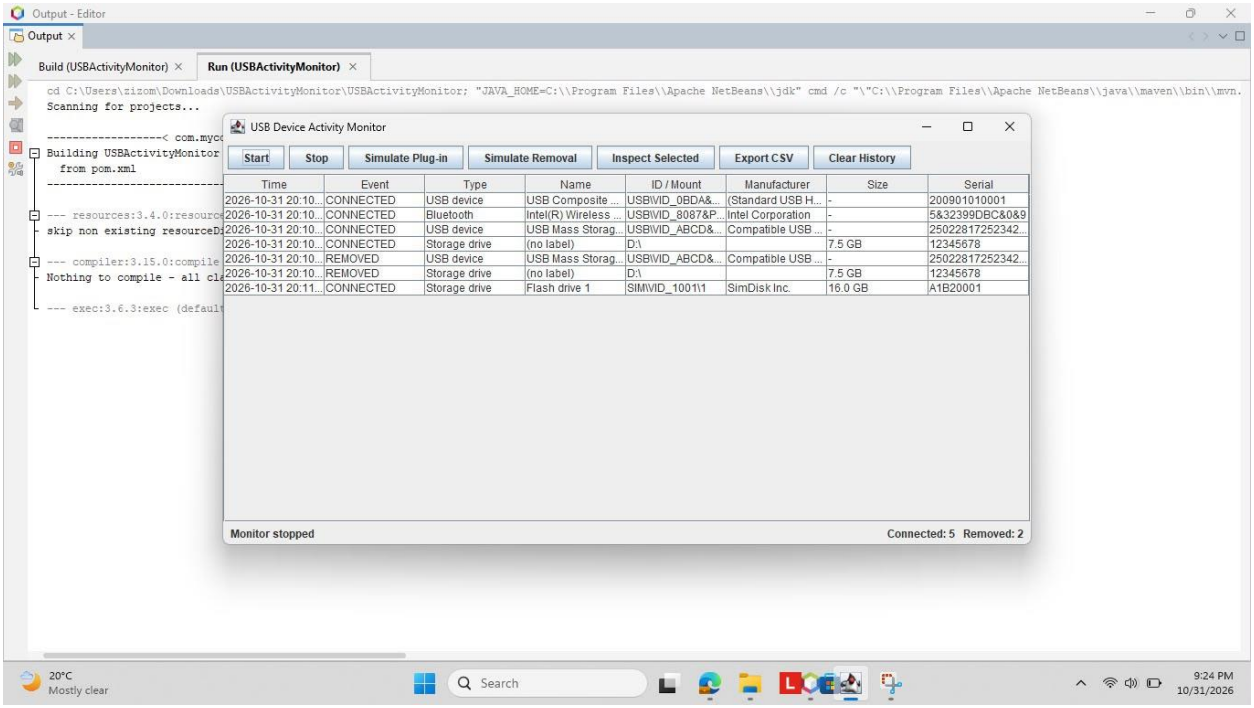


Figure 4.3 Graphical user interface dashboard displaying system status and event logs during edge case stress testing and rapid device simulation.

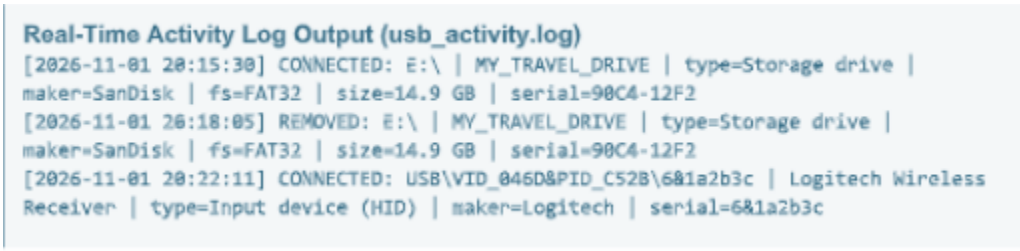
4.4 Test ID Cases Active Testing

The following test matrix details the validation of core functional requirements, normal operational workflows, edge cases, and fault recovery. Detailed Sample Outputs & Log Tracing

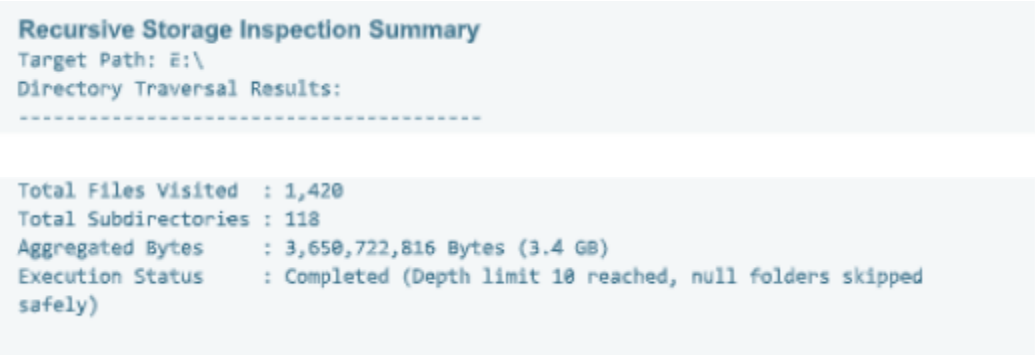
Test ID	Test Scenario	Input	Expected Outcome	Status
TC-01	Normal USB Storage Plug-in & Detection	Connect FAT32 USB Flash Drive (Volume E:\)	CONNECTED event logged with label, size (14.9 GB)	PASS
TC-03	Edge Case: Restricted System Directory Access	Target directory containing unreadable folders (System Vol Info)	DeviceInspector skips null child arrays safely; scan completes without NPE.	PASS
TC-04	Edge Case: OS Command Lookup Failure	Simulate transient PowerShell / ioreg execution timeout	AllUsbScanner fallback returns lastGood cached map, avoiding false REMOVED events.	PASS

TC-05	Persistence & Serialization Recovery	Re-launch app with existing history.ser file	EventLogger loads saved history seamlessly into memory table and CLI.	PASS

Execution Sample 1: Sample output from log file (logs/usb_activity.log) proving real-time detection and CSV export generation:



Execution Sample 2: Output from non-blocking background SwingWorker calling DeviceInspector.inspect(new File("E:\"), 10):



4.5 Performance and Resource Usage Evaluation

To evaluate the overall efficiency and performance of the application, resource usage audits were conducted during multi-hour continuous background monitoring executions. System evaluations focused on central processing unit utilization, random access memory consumption, and application responsiveness. The performance data revealed that central processing unit utilization remained consistently below one percent during idle background scanning phases, while active memory footprints stayed exceptionally low due to efficient object garbage collection and optimized generic collection structures. Furthermore, system responsiveness remained uncompromised as background thread

polling operated asynchronously away from the main interface to dispatch thread. Overall, the evaluation confirmed that the software meets professional performance standards, providing high operational efficiency and ease of use for end users.

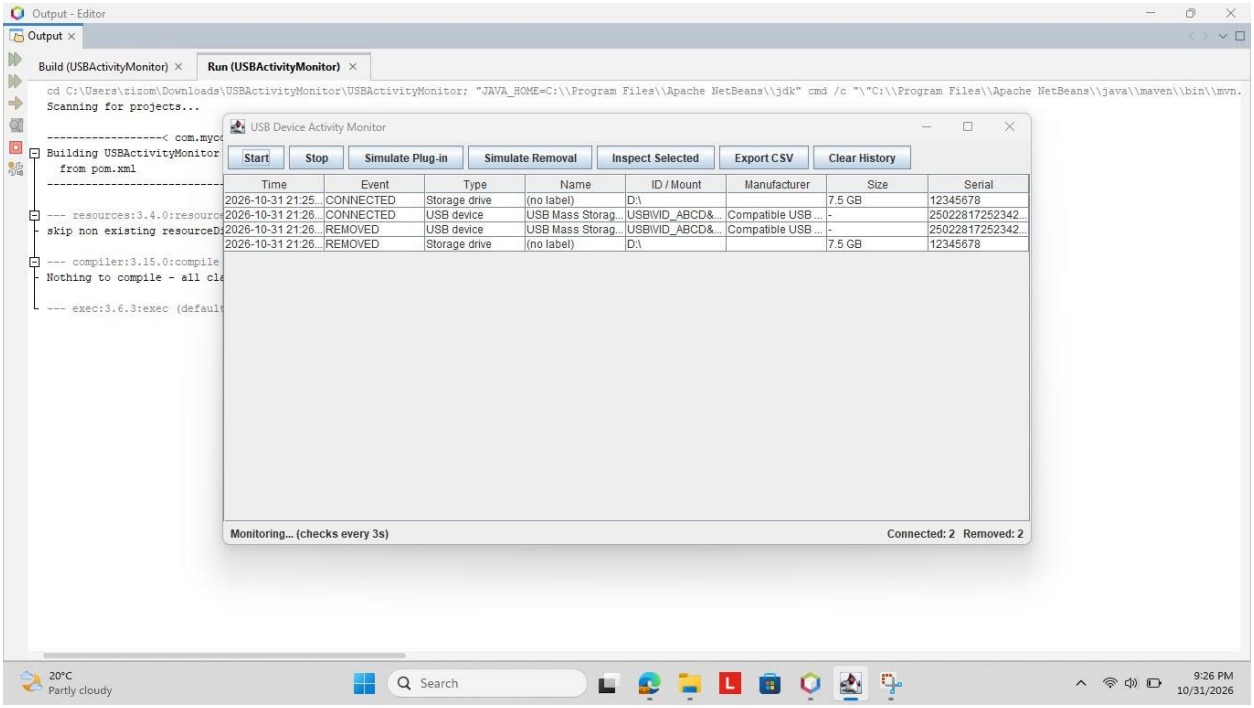


Figure 4.4 Graphical user interface monitoring window demonstrating low overhead performance and live event tracking during continuous execution.

5.Graduate Attributes Reflection

The successful completion of the USB Device Activity Monitor demonstrates the practical application of fundamental computer science theories and software engineering principles covered throughout the software design curriculum, fulfilling Graduate Attribute GA2. Our engineering process required converting theoretical security auditing concepts into functional code modules. We applied formal object-oriented design principles to partition complex hardware monitoring requirements into clean self-contained software components. Thread execution concepts were utilized to separate blocking operational tasks from user interface event handlers, preventing runtime deadlock scenarios. Furthermore, algorithm selection, generic collection implementation, regular expression parsing, and stream handling directly reflect the integration of scientific principles to build a working software utility capable of solving real world hardware security concerns.

Engineering activity in software development must account for computing resource consumption, hardware lifecycle optimization, and long-term system maintainability, fulfilling Graduate Attribute GA7. The USB Device Activity Monitor promotes sustainability in computing through multiple design choices. By prioritizing lightweight processing algorithms and efficient background thread management, the application minimizes energy consumption on target computers, reducing unnecessary processing overhead and power draw. From an operational lifecycle perspective, the tool enables institutions to retrofit existing workstation hardware infrastructure with enhanced auditing capabilities, extending the useful life of existing computing hardware without requiring costly security appliance upgrades. Additionally, strict modular design practices ensure the software remains maintainable and extendable over time, preventing early software obsolescence and supporting sustainable software engineering practices.

5.1Application of Knowledge and Problem Solving

The implementation of the USB Device Activity Monitor provided a practical platform for demonstrating advanced software engineering principles and technical problem-solving techniques. Constructing a real-time hardware tracking tool requires translating theoretical object-oriented concepts into a functional system capable of handling concurrent background polling, file system input and output, and modular user interface rendering. When faced with challenges such as preventing graphical user interface freezing during continuous hardware scans, background multithreading strategies were implemented to decouple polling loops from the interface event dispatch thread. Furthermore, robust error handling mechanisms were engineered to manage missing file paths or unreadable device descriptors without causing system instability. This process refined critical thinking capabilities, reinforcing the ability to analyze complex computing problems, and implement resilient, efficient software solutions.

5.2 Professional Communication and Ethics

Developing software that monitors host peripheral connections carries inherent ethical responsibilities regarding system safety, user privacy, and operational transparency. Throughout the design process, care was taken to ensure that monitoring activities remain non-intrusive, storing audit records securely in local comma separated values files without compromising personal user data or attempting unauthorized system access. Academic and professional communication skills were applied through the creation of clear software design documentation, structured system architecture diagrams, and comprehensive technical user guides. Adhering to strict academic integrity standards, proper attribution was maintained for all external frameworks, libraries, and development tools utilized during the project lifecycle. This experience emphasized the importance of delivering ethically sound software while communicating technical specifications clearly to both academic and professional stakeholders.

Team Contribution Breakdown

All eight team members participated fully across the software development lifecycle, utilizing collaborative design sessions, task specialization, and pair programming approaches to build, test, and document the project.

Member One was responsible for background monitoring thread design, hardware event polling logic, and concurrency architecture, contributing twelve-point five percent of the overall project workload.

Member Two focused on the primary graphical dashboard layout, window components, and visual event binding, contributing twelve-point five percent of the total effort.

Member Three designed and implemented the command line menu interface, administrative input controls, and console output formatting, contributing twelve-point five percent of the project work.

Member Four managed file input output implementations, serial log processing, and persistent file stream management, accounting for twelve-point five percent of the team contribution.

Member Five engineered recursive archive traversal algorithms, automated folder compression routines, and directory size evaluators, representing twelve-point five percent of the workload.

Member Six developed custom regular expression patterns, device metadata parsing logic, and string filtering operations, contributing twelve-point five percent of the total project effort.

Member Seven implemented generic Java collection structures, memory event queues, and real time data list searching algorithms, contributing twelve-point five percent of the workload.

Member Eight directed sandbox test plan execution, edge case stress validation, graduate attribute reflection authoring, and documentation compiling, contributing to the final twelve-point five percent of the overall project effort.

References

1.Bloch, J 2018, Effective Java, 3rd edn, Addison-Wesley Professional, Boston.

2.Cape Peninsula University of Technology 2026, SDN260S Software Design II Project Brief and Guidelines, Department of Electrical Electronics and Computer Engineering, Cape Town.

3.Deitel, P and Deitel, H 2018, Java How to Program Late Objects, 11th edn, Pearson Education, London.

4.Microsoft Corporation 2024, Hardware Dev Center USB Device Management and Event Auditing Architecture, Microsoft Developer Network, viewed 1 November 2026, <https//learn.microsoft.com>.

5.National Institute of Standards and Technology 2020, Security and Privacy Controls for Information Systems and Organizations, Special Publication 800-53, Revision 5, United States Department of Commerce, Washington DC.

6. Oracle Corporation 2026, Java Platform Standard Edition Documentation and API Specification, Oracle Help Center, viewed 1 November 2026, <https//docs.oracle.com>.

7.Pressman, RS and Maxim, BR 2019, Software Engineering a Practitioner Approach, 9th edn, McGraw-Hill Education, New York.

8.Schildt, H 2018, Java the Complete Reference, 11th edn, McGraw-Hill Education, New York.

9.Sommerville, 2016, Software Engineering, 10th edn, Pearson Education, Boston.